EMBEDDED OPERATIVNI SISTEMI

Linux drajver i korisnička aplikacija za NCC akcelerator

Druga verzija, usklađena sa vežbama

Grupa: **y25-g10**

Nastavak projekta sa predmeta Projektovanje složenih digitalnih sistema

Sadržaj

1. Uvod

2. Hardverski interfejs akceleratora

3. Arhitektura rešenja

4. Platformski drajver

4.1. Učitavanje i uklanjanje modula

4.2. Probe i remove

4.3. Operacije nad uređajem

4.4. Komande

4.5. Sinhronizacija

4.6. Mapiranje memorije

4.7. Veza sa vežbama

5. Korisnička aplikacija

6. Prevođenje i pokretanje

6.1. Device tree

6.2. Drajver

6.3. Aplikacija

6.4. Provera na ploči

7. Rezultati

8. Zaključak

1. Uvod

Predmet rada je Linux podrška za IP jezgro `ncc_accel`: hardverski akcelerator normalizovane unakrsne korelacije (NCC), projektovan na predmetu Projektovanje složenih digitalnih sistema. Akcelerator se koristi za prepoznavanje šahovskih figura: slika table deli se na 64 polja, a svako zauzeto polje poredi se sa šablonima figura. Rezultat je pozicija u FEN zapisu.

U prethodnoj fazi sistem je radio kao bare-metal aplikacija u Vitis-u, koja registrima i memorijama akceleratora pristupa direktno, funkcijama `Xil_Out32` i `Xil_In32`. Ovde se isti hardver, bez izmene bitstream-a, koristi iz Linux-a: kroz platformski drajver koji upravlja akceleratorom i korisničku aplikaciju koja preko drajvera obavlja isto prepoznavanje table.

Ovo je druga verzija rešenja. Prva verzija koristila je mehanizme koji nisu obrađeni na vežbama (`ioctl`, `devm_*` funkcije, `mutex`, prozivanje u kernelu) i imala je 18 fajlova. Nova verzija koristi isključivo ono što je obrađeno na vežbama: komande preko `write`, status preko `read`, podatke preko `mmap`, i ima 6 fajlova.

Stavka	Vrednost
Ploča	Digilent Zybo Z7-10 (Zynq-7010, `xc7z010clg400-1`)
Hardver	isti `ncc_system_wrapper.xsa` kao u bare-metal rešenju: dve instance `ncc_accel`
Distribucija	PetaLinux 2025.2
Kernel	linux-xlnx 6.12.40, ARM32
Korenski fajl-sistem	initrd (u radnoj memoriji)
Konzola	UART, 115200 bit/s

Tabela 1. *Platforma i alati*

2. Hardverski interfejs akceleratora

Sistem ima dve nezavisne instance akceleratora. Svaka ima dva AXI interfejsa: AXI4-Lite (S00) za kontrolne registre i AXI4-Full (S01) za memorije slike, šablona i rezultata (Tabela 2).

Instanca	S00 (registri)	S01 (memorije)	Linux uređaj
`ncc0`	`0x5000_0000`, 4 KB	`0x5002_0000`, 128 KB	`/dev/ncc0`
`ncc1`	`0x5100_0000`, 4 KB	`0x5102_0000`, 128 KB	`/dev/ncc1`

Tabela 2. *Adresna mapa instanci*

Adresna magistrala S00 široka je šest bita, pa su od prozora od 4 KB važeće samo adrese 0x00–0x3F. Drajver koristi šest registara (Tabela 3).

Adresa	Registar	Opis
`0x00`	`IMG_W`	širina slike, 1–90
`0x04`	`IMG_H`	visina slike, 1–90
`0x08`	`TMP_W`	širina šablona, 1–30
`0x0C`	`TMP_H`	visina šablona, 1–30
`0x30`	`CTRL`	upis 1 pokreće obradu i briše done
`0x34`	`STATUS`	bit 0 = done (ostaje 1 do sledećeg starta), bit 1 = busy; samo čitanje

Tabela 3. *Kontrolni registri (S00)*

Oblast	Pomeraj	Kapacitet	Upotreba
slika	`+0x00000`	8192 reči	do 90×90 = 8100 piksela
šablon	`+0x08000`	1024 reči	do 30×30 = 900 piksela
rezultat	`+0x10000`	8192 reči	NCC² po poziciji prozora, samo za čitanje

Tabela 4. *Oblasti memorijskog prostora S01*

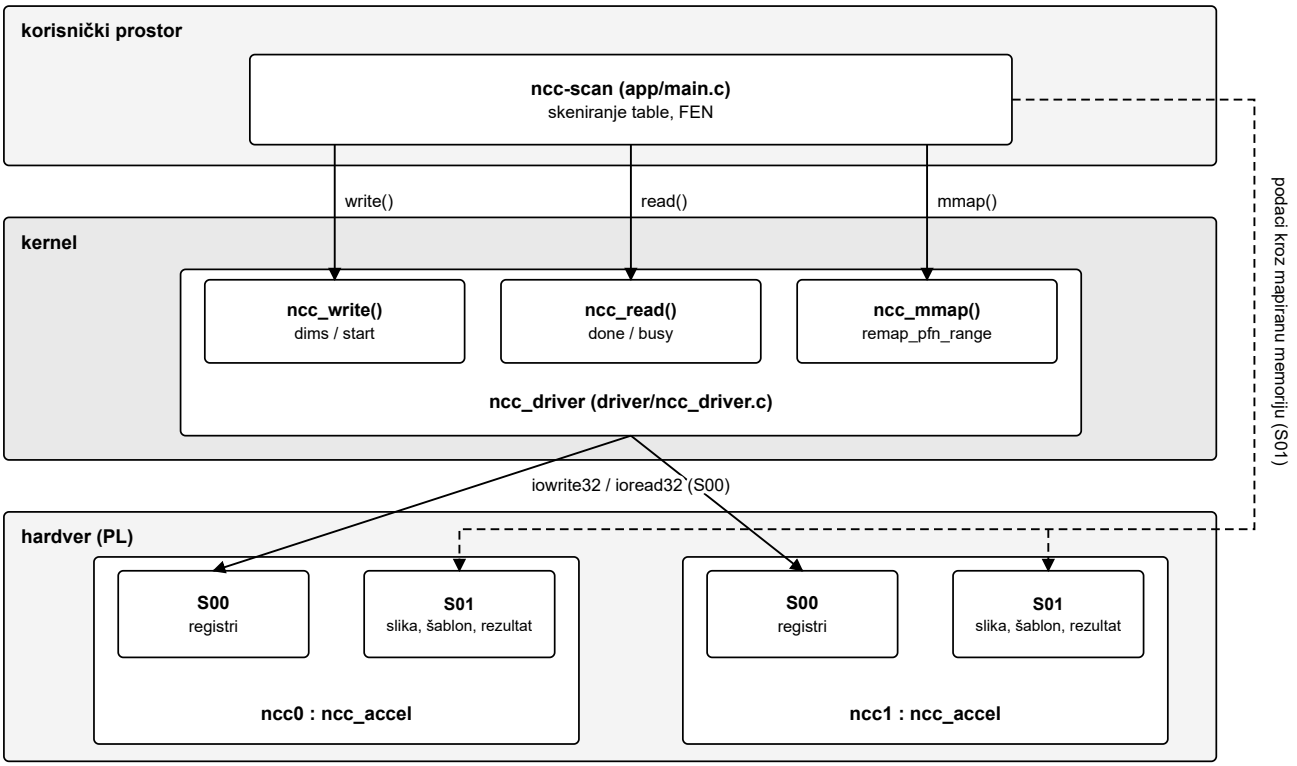
Jedan piksel zauzima jednu 32-bitnu reč, a ne jedan bajt. Rezultat je NCC² u formatu Q1.31, gde `0x80000000` predstavlja 1,0, odnosno savršeno poklapanje. Zato se rezultati porede kao neoznačeni brojevi: kao označen broj, najbolji mogući rezultat bio bi negativan. Akcelerator nema prekidni izlaz, pa se kraj obrade prepoznaje samo po bitu `done` u registru `STATUS`.

Iz verifikacije na predmetu FVH poznato je još jedno ponašanje koje softver mora da poštuje: upis u `CTRL` dok jezgro radi jezgro ignoriše, a magistrala ipak odgovara da je upis uspeo. Drajver zato pre svake komande proverava bit `busy`.

3. Arhitektura rešenja

Rešenje ima tri sloja: korisničku aplikaciju, kernel drajver i hardver (slika 1). Svaka instanca akceleratora je jedan uređaj u `/dev`. Kontrolni i podatkovni put su razdvojeni:

- **kontrola** — tekstualne komande upisom u uređaj (`write`), kao komande za LED u vežbi 9;
- **status** — čitanjem uređaja (`read`) dobija se tekst `done=1 busy=0`;
- **podaci** — memorija S01 se mapira u adresni prostor aplikacije (`mmap`), kao u vežbi 12, pa aplikacija sliku i šablon upisuje, a rezultate čita, kao običan niz.



Slika 1. *Arhitektura rešenja. Komande i status idu kroz drajver; podacima u S01 aplikacija pristupa direktno, kroz mapiranu memoriju (isprekidane linije), bez kopiranja kroz kernel.*

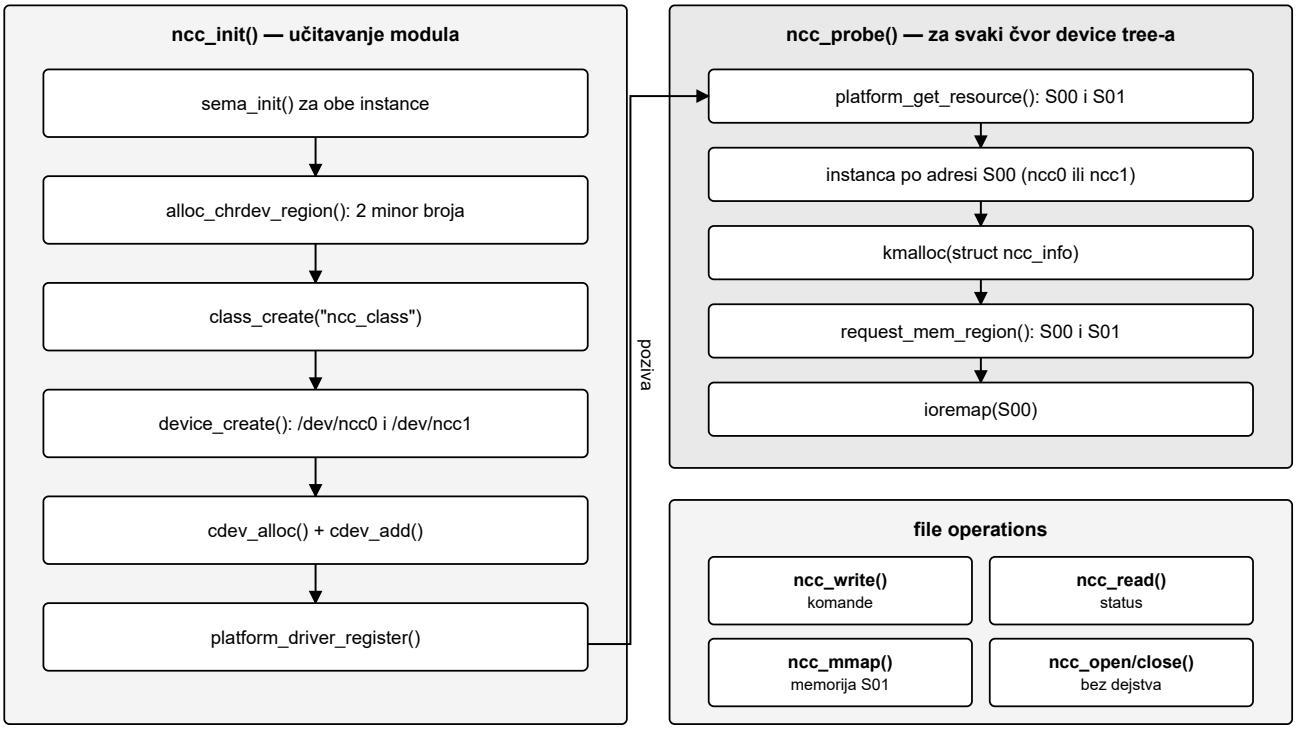
Fajlovi rešenja dati su u tabeli 5.

Fajl	Sadržaj	Linija
`driver/ncc_driver.c`	platformski drajver za obe instance	364
`driver/Makefile`	prevođenje modula (isti oblik kao u vežbi 9)	22
`app/main.c`	aplikacija: rad sa drajverom, obrada slike, skeniranje table, FEN	342
`app/data.c`	slika table 720×720 i 12 šablona u dve rezolucije (generisani podaci)	33 194
`app/data.h`	deklaracije podataka	11
`app/Makefile`	prevođenje aplikacije	12

Tabela 5. *Fajlovi rešenja*

4. Platformski drajver

Drajver je jedan kernel modul, `ncc_driver.ko`, napisan po uzoru na drajvere iz vežbi 6, 7, 9 i 10. Njegova struktura prikazana je na slici 2.



Slika 2. *Struktura drajvera:* `ncc_init()` *pravi uređaje i registruje platformski drajver, kernel zatim poziva* `ncc_probe()` *za obe instance iz device tree-a.*

4.1. Učitavanje i uklanjanje modula

`ncc_init()` rezerviše dva broja uređaja (`alloc_chrdev_region` sa dva minora), pravi klasu i dva čvora, `/dev/ncc0` i `/dev/ncc1`, kao drajver sa više minor brojeva iz vežbe 6. Zatim pravi jedan `cdev` za oba minora i registruje platformski drajver. Ako neki korak ne uspe, prethodni se poništavaju redom (`goto fail_N`). `ncc_exit()` radi isto obrnutim redom.

Dve stvari se razlikuju od vežbi zbog novijeg kernela (6.12 u PetaLinux-u 2025.2): `class_create()` prima samo ime klase (bez `THIS_MODULE`), a funkcija `remove` platformskog drajvera vraća `void` umesto `int`.

4.2. Probe i remove

Kernel poziva `ncc_probe()` jednom za svaki čvor device tree-a sa `compatible = "xlnx,ncc-accel-1.0"`. Svaki čvor ima dva opsega: `reg[0]` je S00, a `reg[1]` je S01. Probe:

- čita oba opsega sa `platform_get_resource()`;
- određuje instancu po fizičkoj adresi S00 (0x5000_0000 je ncc0, 0x5100_0000 je ncc1), jer redosled poziva probe nije garantovan;
- pravi strukturu `ncc_info` sa `kmalloc` i čuva adrese oba opsega;
- rezerviše oba opsega sa `request_mem_region()` i mapira S00 sa `ioremap()`, kao u vežbama 9 i 11;

- S01 se ne mapira u kernelu — mapira ga `ncc_mmap()` direktno u adresni prostor aplikacije.

`ncc_remove()` oslobađa sve obrnutim redom: `iounmap`, `release_mem_region` za oba opsega i `kfree`.

4.3. Operacije nad uređajem

Sve operacije dobijaju instancu iz minor broja otvorenog fajla, `MINOR(pfile->f_inode->i_rdev)`, isto kao drajver sa više minora iz vežbe 6 (Tabela 6).

Funkcija	Šta radi
`ncc_write()`	prima komandu (`copy_from_user`), proverava je i upisuje registre (`iowrite32`)
`ncc_read()`	čita `STATUS` (`ioread32`) i vraća tekst `done=D busy=B`; drugo čitanje istog otvorenog fajla vraća 0, pa radi i `cat`
`ncc_mmap()`	mapira 128 KB memorije S01 bez keša (`pgprot_noncached`, `remap_pfn_range`)
`ncc_open()`	bez dejstva
`ncc_close()`	bez dejstva

Tabela 6. *Operacije nad uređajem*

4.4. Komande

Komande su tekstualne, kao komande za LED u vežbi 9, pa se drajver može koristiti i iz shell-a (Tabela 7). Tekst se rastavlja sa `sscanf`.

Komanda	Dejstvo	Greška
`dims IW IH TW TH`	upisuje dimenzije u `IMG_W`, `IMG_H`, `TMP_W`, `TMP_H`	`EINVAL` ako je slika van 1–90, šablon van 1–30 ili šablon veći od slike
`start`	upisuje 1 u `CTRL`: pokreće obradu i briše done	—
(bilo koja, dok je busy = 1)	ništa se ne upisuje	`EBUSY`
(nepoznata komanda)	ništa se ne upisuje	`EINVAL`
čitanje (`cat /dev/ncc0`)	vraća `done=1 busy=0` ili sl.	—

Tabela 7. *Komande drajvera*

Provera `busy` je posledica nalaza N2 iz verifikacije: jezgro start dok radi ignoriše, a nove dimenzije bi promenile posao koji je u toku. Bez provere aplikacija bi dobila uspešan odgovor na komandu koja nije izvršena i pročitala bi rezultate prethodnog posla.

4.5. Sinhronizacija

Komanda se izvršava pod semaforom svoje instance (`down_interruptible` / `up`, vežba 7). Tako dva procesa koja istovremeno šalju komande istom akceleratoru ne mogu da pomešaju upise četiri registra dimenzija. `down_interruptible` vraća grešku ako proces dobije signal dok čeka, pa se proces može prekinuti.

4.6. Mapiranje memorije

`ncc_mmap()` dozvoljava mapiranje do 128 KB (ceo S01) i postavlja stranice kao nekeširane. S01 je memorija akceleratora iza AXI magistrale, a ne radna memorija: sa kešom bi upisi procesora mogli da ostanu u kešu i ne stignu do akceleratora, a čitanja bi mogla da vrate staru vrednost umesto rezultata. Za razliku od vežbe 12, gde se mapira DMA bafer u radnoj memoriji (`dma_mmap_coherent`), ovde se mapira memorija uređaja, pa se koristi `remap_pfn_range`.

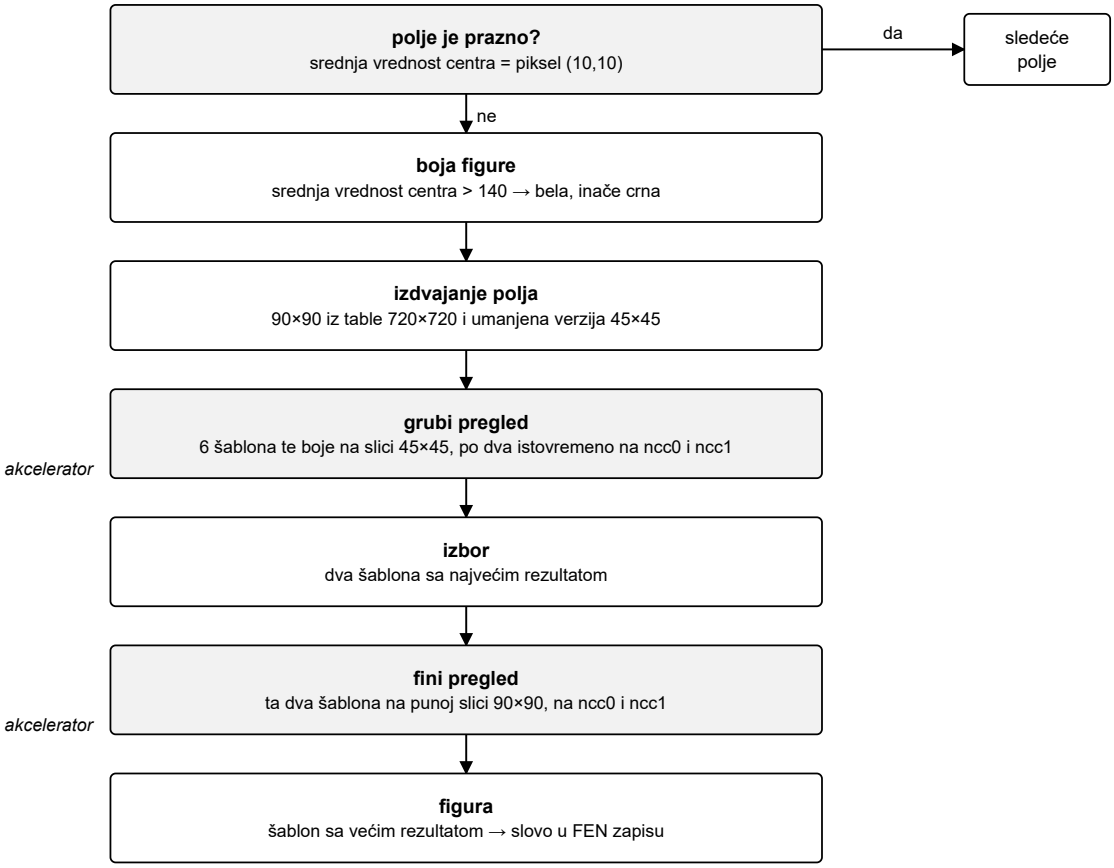
4.7. Veza sa vežbama

Mehanizam	Vežba
`alloc_chrdev_region`, `class_create`, `device_create`, `cdev_alloc`, `cdev_add`	5, 6
više minor brojeva, `MINOR(...)`	6
`read` / `write`, `copy_to_user` / `copy_from_user`, `sscanf`	6, 9
semafor: `sema_init`, `down_interruptible`, `up`	7
platformski drajver, `of_match_table`, `platform_get_resource`	9, 10
`kmalloc`, `request_mem_region`, `ioremap`, `ioread32` / `iowrite32`	9, 10, 11
`mmap`	12
PetaLinux, device tree, Linux na Zybo ploči	11, 12
aplikacija koja koristi uređaj kroz fajl u /dev	3

Tabela 8. *Mehanizmi drajvera i vežbe u kojima su obrađeni*

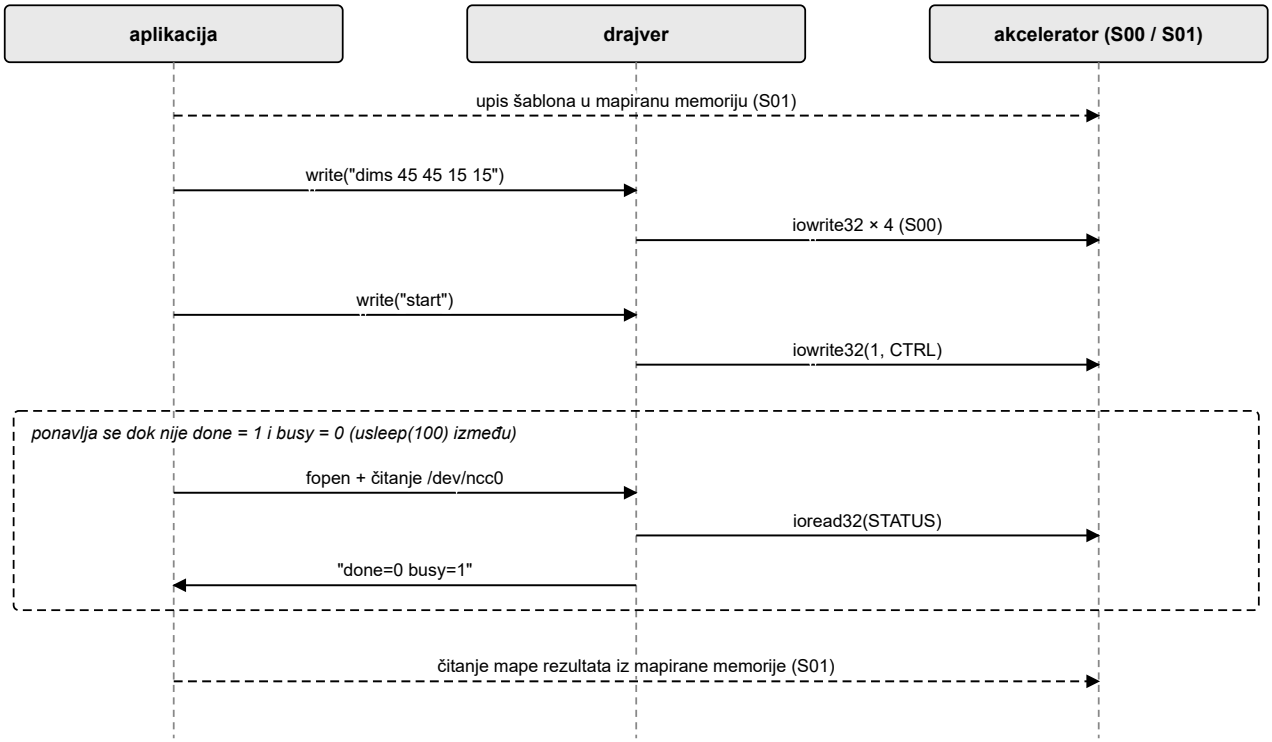
5. Korisnička aplikacija

Aplikacija `ncc-scan` obrađuje istu tablu i iste šablone kao bare-metal rešenje: 64 polja od 90×90 piksela i 12 šablona (šest belih i šest crnih figura). Algoritam za jedno polje prikazan je na slici 3.



Slika 3. *Prepoznavanje figure na jednom polju table.*

Svaki par šablona obrađuje se funkcijom `par()`, čiji je tok prikazan na slici 4: šablon se upisuje u mapiranu memoriju, dimenzije i start šalju se drajveru, aplikacija čeka kraj obrade i čita najveći rezultat iz mapirane memorije.



Slika 4. *Tok obrade jednog šablona na jednom akceleratoru. Isprekidane linije: direktan pristup memoriji S01 kroz mmap.*

Oba akceleratora se pokreću pre čekanja, pa rade istovremeno. Aplikacija uvek čeka oba, čak i kada jedan javi grešku, da sledeći par ne bi pisao u memoriju jezgra koje još radi. Funkcije aplikacije date su u tabeli 9.

Funkcija	Šta radi
`ncc_otvori()`	otvara `/dev/nccN` i mapira 128 KB memorije S01
`ncc_komanda()`	šalje tekstualnu komandu drajveru (`write`)
`ncc_dimenzije()`	pravi komandu `dims IW IH TW TH`
`ncc_cekaj()`	čita status (`fopen` + `fscanf`) dok nije done = 1 i busy = 0; najviše oko 2 s
`upisi()`	upisuje piksele (8 bita) kao 32-bitne reči u mapiranu memoriju
`najbolji_skor()`	najveći rezultat u mapi, poređenje kao neoznačeni broj
`par()`	jedan ili dva šablona na ncc0 i ncc1 nad već upisanom slikom
`umanji()`	umanjenje slike 2× (prosek 2×2 piksela, sa zaokruživanjem)
`srednja_centra()`	srednja vrednost centralnih 31×31 piksela polja
`prazno()`	da li je polje prazno
`skeniraj_polje()`	ceo algoritam sa slike 3 za jedno polje
`napravi_fen()`	tabla 8×8 u FEN zapis
`main()`	otvaranje uređaja, skeniranje 64 polja, ispis i poređenje FEN-a; vraća 1 ako se FEN ne poklapa

Tabela 9. *Funkcije u `app/main.c`*

6. Prevođenje i pokretanje

6.1. Device tree

PetaLinux iz `.xsa` fajla sam generiše čvorove za obe instance, sa dva `reg` opsega i bez `interrupts`, jer IP nema prekidni izlaz:

```
ncc_accel@50000000 {
    compatible = "xlnx,ncc-accel-1.0";
    reg = <0x50000000 0x1000>, <0x50020000 0x20000>;
};
ncc_accel@51000000 {
    compatible = "xlnx,ncc-accel-1.0";
    reg = <0x51000000 0x1000>, <0x51020000 0x20000>;
};
```

6.2. Drajver

Modul se prevodi protiv istog kernela koji radi na ploči. U PetaLinux projektu to radi recept modula (`project-spec/meta-user/recipes-modules/`); ime recepta mora imati crticu, a izvorni fajl u receptu zamenjuje se sa `ncc_driver.c`. Van PetaLinux-a:

```
cd driver
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- KERNELDIR=<kernel build dir>
```

6.3. Aplikacija

```
cd app
make CROSS_COMPILE=arm-linux-gnueabihf-
```

6.4. Provera na ploči

Pošto su komande tekstualne, drajver se proverava iz shell-a, pre pokretanja aplikacije (Tabela 10).

Komanda	Očekivano
`insmod ncc_driver.ko`	u `dmesg`: `ncc_probe: ncc0 …` i `ncc_probe: ncc1 …`
`ls /dev/ncc*`	`/dev/ncc0` i `/dev/ncc1`
`cat /dev/ncc0`	`done=0 busy=0` posle uključenja
`echo "dims 4 4 2 2" > /dev/ncc0`	bez greške
`echo "dims 100 4 2 2" > /dev/ncc0`	greška `Invalid argument`
`echo "start" > /dev/ncc0`	bez greške
`cat /dev/ncc0`	`done=1 busy=0`
`./ncc-scan`	FEN se poklapa sa očekivanim, izlazni kod 0

Tabela 10. *Provera drajvera iz shell-a*

7. Rezultati

Ispravnost aplikacije proverena je na računaru, pre prenosa na ploču: `main.c` je preveden zajedno sa softverskim modelom akceleratora umesto drajvera. Model prima iste komande (`dims`, `start`), računa NCC² istom aritmetikom kao hardver (isti model je u FVH okruženju bit-tačan prema RTL-u) i vraća status `done=1 busy=0`. Rezultat (Tabela 11): svih 32 zauzetih polja prepoznato je tačno.

```
zauzetih polja: 32
FEN:       rnbqkbnr/pp5p/4ppp1/2pp4/5P2/1P1BPN2/P1PPQ1PP/RNB1K2R
ocekivano: rnbqkbnr/pp5p/4ppp1/2pp4/5P2/1P1BPN2/P1PPQ1PP/RNB1K2R
Rezultat se poklapa sa ocekivanom pozicijom.
```

Provera	Rezultat
aplikacija sa softverskim modelom (računar, gcc -Wall)	**FEN tačan, 32 / 32 polja**
prevođenje drajvera (PetaLinux, kernel 6.12)	*čeka*
provera iz shell-a na ploči (Tabela 10)	*čeka*
`ncc-scan` na ploči	*čeka*

Tabela 11. *Rezultati provere*

Za poređenje, prva verzija (sa `ioctl`) dala je na ploči isti tačan FEN za 1,906 s, prema 1,782 s bare-metal rešenja (Tabela 12). Skoro cela razlika bila je u čekanju na akcelerator: drajver je između provera statusa spavao 0,5–1,5 ms, a aplikacija čeka oko 128 puta po tabli. Nova verzija proziva status iz aplikacije svakih 100 µs, pa se očekuje vreme bliže bare-metal rešenju; tačna vrednost upisuje se posle merenja na ploči.

	Bare-metal (Vitis)	Linux, prva verzija (ioctl)	Linux, druga verzija
Ukupno vreme	1,782 s	1,906 s	*meri se*
Čekanje na akcelerator	1,555 s	1,677 s	*meri se*
Prepoznata polja	32 / 32	32 / 32	*meri se*

Tabela 12. *Vreme prepoznavanja cele table (prethodna merenja)*

8. Zaključak

Isti hardver koji je u PSDS-u radio kao bare-metal sistem, a u FVH-u verifikovan u simulaciji, koristi se iz Linux-a kroz platformski drajver i korisničku aplikaciju. Drajver koristi samo mehanizme obrađene na vežbama: tekstualne komande kroz `write`, status kroz `read`, memoriju akceleratora kroz `mmap`, semafor za sinhronizaciju i `kmalloc` / `request_mem_region` / `ioremap` za resurse. Rešenje ima šest fajlova. Drajver odbija komande dok jezgro radi, što direktno sprečava grešku otkrivenu verifikacijom (nalaz N2).

Aplikacija je na računaru, sa softverskim modelom akceleratora, prepoznala svih 32 figure. Preostaje provera drajvera i aplikacije na ploči i novo merenje vremena.